LAB 7 – Joins

Inner Join

```
CREATE TABLE Department (
    dept_id   INT PRIMARY KEY,
    dept_name VARCHAR(30)
);

CREATE TABLE Employee (
    emp_id   INT PRIMARY KEY,
    emp_name VARCHAR(30),
    dept_id  INT
);

CREATE TABLE Staff (
    emp_id     INT PRIMARY KEY,
    emp_name   VARCHAR(30),
    manager_id INT
);
```

```
INSERT INTO Department VALUES (10, 'Sales'), (20, 'HR'), (30, 'IT'), (40, 'Finance');

INSERT INTO Employee VALUES
(1, 'Anika', 10),
(2, 'Rahul', 20),
(3, 'Maya',  30),
(4, 'Joel',  NULL);

INSERT INTO Staff VALUES
(1, 'Anika', NULL),
(2, 'Rahul', 1),
(3, 'Maya',  1),
(4, 'Joel',  2);
```

```
SELECT e.emp_id, e.emp_name, d.dept_name
FROM Employee e
INNER JOIN Department d ON e.dept_id = d.dept_id;
```

emp_id	emp_name	dept_name
1	Anika	Sales
2	Rahul	HR
3	Maya	IT

Left Join

```
SELECT e.emp_id, e.emp_name, d.dept_name
FROM Employee e
LEFT JOIN Department d ON e.dept_id = d.dept_id;
```

emp_id	emp_name	dept_name
1	Anika	Sales
2	Rahul	HR
3	Maya	IT
4	Joel	NULL

Right Join

```
SELECT e.emp_id, e.emp_name, d.dept_name
FROM Employee e
RIGHT JOIN Department d ON e.dept_id = d.dept_id;
```

emp_id	emp_name	dept_name
1	Anika	Sales
2	Rahul	HR
3	Maya	IT
NULL	NULL	Finance

Fully Outer

```
SELECT e.emp_id, e.emp_name, d.dept_name
FROM Employee e
LEFT JOIN Department d ON e.dept_id = d.dept_id
UNION
SELECT e.emp_id, e.emp_name, d.dept_name
FROM Employee e
RIGHT JOIN Department d ON e.dept_id = d.dept_id;
```

emp_id	emp_name	dept_name
1	Anika	Sales
2	Rahul	HR
3	Maya	IT
4	Joel	NULL
NULL	NULL	Finance

Cross Join

```
SELECT e.emp_name, d.dept_name
FROM Employee e
CROSS JOIN Department d;
```

emp_name	dept_name
Joel	Sales
Maya	Sales
Rahul	Sales
Anika	Sales
Joel	HR
Maya	HR
Rahul	HR
Anika	HR
Joel	IT
Maya	IT
Rahul	IT
Anika	IT
Joel	Finance
Maya	Finance
Rahul	Finance
Anika	Finance

Self-Join

```
SELECT e.emp_name AS employee, m.emp_name AS manager
FROM Staff e
INNER JOIN Staff m ON e.manager_id = m.emp_id;
```

employee	manager
Rahul	Anika
Maya	Anika
Joel	Rahul

LAB 8 – Group by, Having and Order by Clause

Group by

```
CREATE TABLE Emp (
    emp_id   INT PRIMARY KEY,
    emp_name VARCHAR(30),
    dept     VARCHAR(30),
    salary   INT
);

INSERT INTO Emp VALUES
(1, 'Anika',  'Sales', 50000),
(2, 'Rahul',  'HR',    40000),
(3, 'Maya',   'IT',    60000),
(4, 'Joel',   'IT',    55000),
(5, 'Pooja',  'Sales', 45000),
(6, 'Karan',  'HR',    42000),
(7, 'Shreya', 'IT',    65000);

SELECT dept,
       COUNT(*)    AS total_emp,
       SUM(salary) AS total_salary,
       AVG(salary) AS avg_salary
FROM Emp
GROUP BY dept;
```

dept	total_emp	total_salary	avg_salary
Sales	2	95000	47500.0000
HR	2	82000	41000.0000
IT	3	180000	60000.0000

Having by

```
SELECT dept, COUNT(*) AS total_emp
FROM Emp
GROUP BY dept
HAVING COUNT(*) > 2;
```

dept	total_emp
IT	3

Order by

```
SELECT emp_name, dept, salary
FROM Emp
ORDER BY salary DESC;
```

emp_name	dept	salary
Shreya	IT	65000
Maya	IT	60000
Joel	IT	55000
Anika	Sales	50000
Pooja	Sales	45000
Karan	HR	42000
Rahul	HR	40000

Group by + having + order by

```
SELECT dept, AVG(salary) AS avg_salary
FROM Emp
GROUP BY dept
HAVING AVG(salary) > 45000
ORDER BY avg_salary DESC;
```

dept	avg_salary
IT	60000.0000
Sales	47500.0000

LAB 9 – Procedure

Basic Procedure

```
CREATE TABLE Emp (
    emp_id   INT PRIMARY KEY,
    emp_name VARCHAR(30),
    dept     VARCHAR(30),
    salary   INT
);

INSERT INTO Emp VALUES
(1, 'Anika',  'Sales', 50000),
(2, 'Rahul',  'HR',    40000),
(3, 'Maya',   'IT',    60000),
(4, 'Joel',   'IT',    55000),
(5, 'Pooja',  'Sales', 45000),
(6, 'Karan',  'HR',    42000),
(7, 'Shreya', 'IT',    65000);

DROP PROCEDURE IF EXISTS show_emp;

DELIMITER $$
CREATE PROCEDURE show_emp()
BEGIN
    SELECT * FROM Emp;
END$$
DELIMITER ;

CALL show_emp();
```

emp_id	emp_name	dept	salary
1	Anika	Sales	50000
2	Rahul	HR	40000
3	Maya	IT	60000
4	Joel	IT	55000
5	Pooja	Sales	45000
6	Karan	HR	42000
7	Shreya	IT	65000

Procedure for Insert

```
DELIMITER $$
CREATE PROCEDURE insert_emp(
    IN p_id     INT,
    IN p_name   VARCHAR(30),
    IN p_dept   VARCHAR(30),
    IN p_salary INT
)
BEGIN
    INSERT INTO Emp VALUES (p_id, p_name, p_dept, p_salary);
END$$
DELIMITER ;

CALL insert_emp(8, 'Aditya', 'HR', 38000);
SELECT * FROM Emp;
```

emp_id	emp_name	dept	salary
1	Anika	Sales	50000
2	Rahul	HR	40000
3	Maya	IT	60000
4	Joel	IT	55000
5	Pooja	Sales	45000
6	Karan	HR	42000
7	Shreya	IT	65000
8	Aditya	HR	38000

Procedure for Update

```
DELIMITER $$
CREATE PROCEDURE update_salary(
    IN p_id         INT,
    IN p_new_salary INT
)
BEGIN
    UPDATE Emp
    SET salary = p_new_salary
    WHERE emp_id = p_id;
END$$
DELIMITER ;

CALL update_salary(2, 45000);
SELECT * FROM Emp;
```

emp_id	emp_name	dept	salary
1	Anika	Sales	50000
2	Rahul	HR	45000
3	Maya	IT	60000
4	Joel	IT	55000
5	Pooja	Sales	45000
6	Karan	HR	42000
7	Shreya	IT	65000

Procedure for Delete

```
DELIMITER $$
CREATE PROCEDURE delete_emp(
    IN p_id INT
)
BEGIN
    DELETE FROM Emp
    WHERE emp_id = p_id;
END$$
DELIMITER ;

CALL delete_emp(5);
SELECT * FROM Emp;
```

emp_id	emp_name	dept	salary
1	Anika	Sales	50000
2	Rahul	HR	40000
3	Maya	IT	60000
4	Joel	IT	55000
6	Karan	HR	42000
7	Shreya	IT	65000

LAB 10 – Triggers

```
DROP TRIGGER IF EXISTS before_emp_insert;
DROP TRIGGER IF EXISTS after_emp_insert;
DROP TRIGGER IF EXISTS before_emp_update;
DROP TRIGGER IF EXISTS after_emp_update;
DROP TRIGGER IF EXISTS before_emp_delete;
DROP TRIGGER IF EXISTS after_emp_delete;

CREATE TABLE Emp (
    emp_id   INT PRIMARY KEY,
    emp_name VARCHAR(30),
    dept     VARCHAR(30),
    salary   INT
);

CREATE TABLE Emp_log (
    log_id   INT AUTO_INCREMENT PRIMARY KEY,
    action   VARCHAR(30),
    emp_id   INT,
    log_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO Emp VALUES
(1, 'Anika',  'Sales', 50000),
(2, 'Rahul',  'HR',    40000),
(3, 'Maya',   'IT',    60000),
(4, 'Joel',   'IT',    55000),
(5, 'Pooja',  'Sales', 45000),
(6, 'Karan',  'HR',    42000),
(7, 'Shreya', 'IT',    65000);
```

Triggers

```
DELIMITER $$

CREATE TRIGGER before_emp_insert
BEFORE INSERT ON Emp
FOR EACH ROW
BEGIN
    IF NEW.salary <= 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary must be greater than 0';
    END IF;
END$$

CREATE TRIGGER after_emp_insert
AFTER INSERT ON Emp
FOR EACH ROW
BEGIN
    INSERT INTO Emp_log (action, emp_id) VALUES ('INSERT', NEW.emp_id);
END$$

CREATE TRIGGER before_emp_update
BEFORE UPDATE ON Emp
FOR EACH ROW
BEGIN
    IF NEW.salary < OLD.salary THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Salary cannot be decreased';
    END IF;
END$$

CREATE TRIGGER after_emp_update
AFTER UPDATE ON Emp
```

```
FOR EACH ROW
BEGIN
    INSERT INTO Emp_log (action, emp_id) VALUES ('UPDATE', NEW.emp_id);
END$$

CREATE TRIGGER before_emp_delete
BEFORE DELETE ON Emp
FOR EACH ROW
BEGIN
    IF OLD.salary > 60000 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Cannot delete employee with salary above 60000';
    END IF;
END$$

CREATE TRIGGER after_emp_delete
AFTER DELETE ON Emp
FOR EACH ROW
BEGIN
    INSERT INTO Emp_log (action, emp_id) VALUES ('DELETE', OLD.emp_id);
END$$

DELIMITER ;

INSERT INTO Emp VALUES (8, 'Aditya', 'HR', 38000);
INSERT INTO Emp VALUES (9, 'Deepa',  'IT', 48000);
UPDATE Emp SET salary = 45000 WHERE emp_id = 2;
UPDATE Emp SET salary = 52000 WHERE emp_id = 1;
DELETE FROM Emp WHERE emp_id = 6;
DELETE FROM Emp WHERE emp_id = 5;

SELECT * FROM Emp;
SELECT * FROM Emp_log;
```

log_id	action	emp_id	log_time
1	INSERT	8	2026-10-08 20:28:46
2	INSERT	9	2026-10-08 20:28:46
3	UPDATE	2	2026-10-08 20:28:46
4	UPDATE	1	2026-10-08 20:28:46
5	DELETE	6	2026-10-08 20:28:46
6	DELETE	5	2026-10-08 20:28:46

Emp table after the statements above

emp_id	emp_name	dept	salary
1	Anika	Sales	52000
2	Rahul	HR	45000
3	Maya	IT	60000
4	Joel	IT	55000
7	Shreya	IT	65000
8	Aditya	HR	38000
9	Deepa	IT	48000